



TOHSENO

An Open Protocol for Owned Apple Apps

GENESIS WHITE PAPER

Version 2.0 --- July 28, 2026

Status: Constitutional protocol declaration

Origin: JP Franetovic / Anky, Inc.

First implementation: github.com/jpfraneto/tohseno

*One intention becomes one Apple app.
Every app is born from the same open Fascia.
The Builder controls its identity and history.
Any compatible intelligence may continue its life.*

INTELLIGENCE IMAGINES WITHOUT BOUNDS / MACHINES KEEP THE PROMISES EXACT

"Nodes index. Builders own."

DECLARATION

This paper defines an object that no company, application, server, model, token, or founder is allowed to own.

TOHSENO begins with a simple observation: artificial intelligence can now produce software faster than existing systems can establish what that software is, who controls it, how it continues, and whether another tool can understand it. The missing layer is not more generation. It is continuity.

TOHSENO proposes that one application can exist as a portable, owned protocol object called a **Shot**. A Shot has a permanent identity, a known anatomy, and a signed history controlled by its Builder. It begins locally, can be experienced across Apple surfaces, and can be continued by any compatible Factory without permission from the Factory that produced its first form.

The TOHSENO application built by Anky, Inc. is intended to be the first convincing doorway into this protocol. It is not the official doorway. The first registered Shot may be TOHSENO itself. It receives no authority from being first.

One canonical rulebook. Many independent doorways.
The protocol recognizes valid objects and signatures, not brands.

TOHSENO is written for the Apple ecosystem. The Mac is the workshop. The iPhone, iPad, Vision Pro, and future Apple surfaces are where the consequences of the work are lived. Apple provides the ground; it does not provide protocol identity. An Apple ID may be required by Apple for signing or distribution, but it must never be required to exist in relation to TOHSENO itself.

This paper is constitutional. The exact byte encodings, schemas, conformance vectors, contract source, and deployment addresses belong to a content-addressed **Genesis Bundle**. The paper explains the law. The bundle makes the law executable.

Give every idea a shot. Let reality continue it. Take another one.

Abstract

Artificial intelligence has made code abundant. It has not made software portable, legible, or owned. A generated application normally begins inside a session, inherits an arbitrary structure, and depends on the company or tool that produced it. When the generator changes, the application often loses continuity.

TOHSENO proposes an open protocol for owned Apple applications. Its unit is the **Shot**: one application with a permanent identity, a known anatomy, and a signed history controlled by its **Builder**. A Shot begins locally on a Mac and may be experienced on any Apple surface. It can be understood and continued by another compatible **Factory** without permission from the first.

A **BuilderID** is a stable public controller whose authorized keys may change while the identity remains the same. Each physical device carries its own replaceable **DeviceKey**. Each installation of each generated app receives a separate local **InstallationKey**. Cross-app continuity is explicit and scoped: one app may prove a particular fact to another only after the person authorizes that relationship. No universal tracking identifier, username, password, mandatory TOHSENO account, Apple ID, or shared master secret is required.

All compatible applications share a finite connective structure called the **TOHSENO Apple Fascia**. The purpose and expression of an application may vary without bound; the infrastructure beneath it is finite, inspectable, and pinned. Artificial intelligence may interpret intention and produce open-ended expression inside this structure. Deterministic machinery verifies the promises that must remain exact.

The protocol's public registry witnesses signed actions. It does not create a Shot or grant permission for one to exist. Anyone may relay a valid action; the messenger acquires no authority. Nodes may index, mirror, verify, route, build, test, or sponsor public activity around Shots. The Builder remains the owner.

The first implementation may be built by Anky, Inc. and the first public Shot may be TOHSENO itself. Neither fact grants privilege. The name is not the protocol. The company is not the network. The application is not the doorway that all others must pass through.

Intelligence imagines without bounds; machines keep the promises exact.

Contents

Abstract	3
1 The Problem	7
1.1 Software can now be produced before it can be carried	7
1.2 The new scarce capacity	7
2 The Proposal	8
2.1 Canonical rules, not a canonical application	9
3 The Shot	10
3.1 Taking a Shot	10
3.2 Identity and lineage	10
3.3 Ownership has two material forms	10
3.4 What a Shot is not	11
4 Contact and Evolution	11
4.1 Contact	11
4.2 Two kinds of evidence	12
4.3 Rest	12
5 The BuilderID	12
5.1 A stable identity whose keys may change	12
5.2 The user's authority is not issued by the doorway	13
5.3 Device authorization	13
6 Installation Identity and Consentful Continuity	14
6.1 Continuity is a scoped proof, not a shared secret	14
6.2 Passkeys are convenience, not membership	15
6.3 The possible innovation	15

7	The TOHSENO Apple Fascia	16
7.1	The known anatomy	16
7.2	Finite vocabulary, open expression	16
7.3	The Markdown genome	17
8	The Canonical Objects	18
8.1	The smallest executable constitution	18
9	Signed Actions	19
9.1	A signature needs a shared meaning	19
9.2	Four layers of meaning	19
9.3	The messenger is not the authority	20
10	The Public Witness	20
10.1	The registry witnesses; it does not create	20
10.2	No privileged registrar	21
10.3	The Genesis deployment	21
11	Factories, Nodes, and Doorways	22
11.1	Nodes index. Builders own.	22
11.2	Sponsorship and spam	22
11.3	One doorway may be best without becoming necessary	23
12	Conformance	23
12.1	Application conformance	23
12.2	Protocol record conformance	23
12.3	Artificial intelligence cannot certify itself	24
13	Local First and Privacy	24
13.1	The local artifact is real	24
13.2	Different data belongs in different places	25

13.3	Private by default, public by declaration	25
14	Apple's Ground	25
15	Handles and Economic Objects	26
15.1	A handle is an alias, not an identity	26
15.2	An appcoin may be associated with a Shot	27
15.3	Shot number one	27
16	Permanence and Versioning	27
16.1	The timeless name and the versioned expression	27
16.2	The Genesis Bundle	28
16.3	The paper should remain smaller than its consequences	28
17	Constitutional Invariants	29
18	Conclusion	31
	Glossary	32

I The Problem

I.1 Software can now be produced before it can be carried

For most of computing history, the cost of implementation filtered which ideas were allowed to become software. A person needed technical knowledge, time, money, confidence, and enough social justification to turn a private need into a functioning application. The idea had to become a project before it could become an experience.

Artificial intelligence has broken that sequence. Natural language is becoming a legitimate front door to computation. A person can describe an application and receive substantial working code in minutes.

This opening creates a new bottleneck. Code may arrive before the application has:

- a durable identity;
- a known structure another tool can understand;
- an owner independent of the generating service;
- a history that survives models, devices, and companies;
- a clear boundary between private life and public provenance;
- a way to cooperate with other applications without surrendering identity to an account provider.

Generated software therefore tends to become fragments: chats, previews, arbitrary repositories, demos, abandoned prototypes, and applications that exist only while the original provider remains available. Abundance without continuity becomes debris.

I.2 The new scarce capacity

As implementation becomes cheap, the scarce capacity moves from syntax to judgment. A person must still recognize a coherent intention, give it a complete first form, encounter it in life, learn from the encounter, and decide what should happen next.

The machine can help produce the object. It cannot live the Builder's life or decide what the experience means.

Artificial intelligence expands the range of possible expression. A protocol is needed to preserve identity, ownership, legibility, consent, and continuity across that expansion.

2 The Proposal

TOHSENO is an open protocol for creating and evolving owned Apple applications from a shared set of structural rules.

Its fundamental unit is the **Shot**.

A Shot is one Apple application with a permanent identity, a known anatomy, and a signed history controlled by its Builder.

The protocol is not defined by a particular application, command-line tool, website, company, model, chain interface, or server. Those are entry points and services around the object.

A platform says: *your project exists in our system*. A protocol says: *this object has a form that independent systems can recognize*.

TOHSENO therefore makes two commitments at once:

1. **Infinite expression.** A human intention may lead to an application no template library could predict.
2. **Finite structure.** Every compatible application uses a known, versioned connective anatomy that deterministic tools can inspect.

This is the same productive constraint found in a physical construction system. A finite geometry of joints permits an enormous world of forms. The limitation is not opposed to creativity; it is what allows separate builders, tools, and machines to participate in the same creative world.

The infinite belongs in what the application is for.
The infrastructure beneath it is finite, public, and known.

2.1 Canonical rules, not a canonical application

A protocol requires canonical meaning. It does not require a canonical doorway.

What is canonical:

- the definition of a Shot;
- the authorization model;
- the meaning of signed actions;
- the Apple Fascia;
- the machine schemas;
- the public registry for a deployment;
- the conformance tests.

What is replaceable:

- TOHSENO for Mac;
- TOHSENO Studio;
- a command-line implementation;
- an alternative Factory;
- an intelligence provider;
- a node, relayer, indexer, or paymaster;
- Anky, Inc.;
- the official website.

The bytes of a protocol expression may be canonical.
The intelligences giving those bytes a body are not.

3 The Shot

3.1 Taking a Shot

Taking a Shot is the act of allowing one coherent intention to acquire an independently operable body.

It is more than generating code. An idea inside the mind can remain perfect because reality has not resisted it. A functioning application loses that protection. It may confirm the idea, reveal a deeper one, feel wrong immediately, become useful in an unpredicted way, or show that nothing more should be built. Each is a valid outcome.

A Shot is therefore both a software object and a creative method: one intention receives a real attempt, and the attempt creates evidence.

3.2 Identity and lineage

Every Shot receives a permanent opaque identifier, the **ShotID**. The identifier is not derived from properties that may legitimately change: name, icon, source location, version, model, Factory, bundle identifier, handle, appcoin, or distribution state.

The first accepted record is the **Shot Genesis**. Later meaningful changes are **Evolutions**. Each Evolution points to the previous accepted record, creating an append-only lineage.



Not every commit is an Evolution. An Evolution is a semantic step the Builder recognizes as part of the continuing life of the same application. If the center of the application changes enough that it is best understood as a different object, a new Shot should be taken.

3.3 Ownership has two material forms

A Builder must possess the application as ordinary software: source, build instructions, local history, and the ability to continue without TOHSENO permission.

The Builder must also control the Shot's official protocol lineage through the BuilderID.

Source can be copied. A copy may be valuable and legitimate. Copying source does not silently transfer control of the original Shot. A copied repository may become a fork, an independent application, or the basis for a new Shot. The original lineage continues only through actions authorized by its current controller.

Material ownership: the Builder can leave with the application.

Protocol ownership: the Builder alone authorizes the continuing public lineage.

3.4 What a Shot is not

A Shot is not a prompt, chat session, model response, screenshot, build, commit, repository path, account, token, handle, website, App Store listing, or remote project that disappears when a service ends.

These may describe, finance, display, build, distribute, or witness a Shot. They are not the primitive.

4 Contact and Evolution

4.1 Contact

Contact occurs when a person encounters a functioning form of the Shot in the context where its central behavior can become true or false.

A successful build is machine evidence. It is not yet Contact. Contact begins when the application enters the hands, routines, relationships, or environment for which it was created.

After Contact, the protocol asks one canonical question:

What did using this version teach you about what this application wants to become?

The wording is deliberately different from *what features should be added?* The answer may be that something should disappear, that the application should become smaller, that the real need is adjacent to the initial idea, or that the Shot should rest.

4.2 Two kinds of evidence

A Shot produces an application and evidence. Evidence has two distinct forms.

Machine Evidence establishes bounded technical facts: a schema parsed, a signature verified, a source commitment matched, a conformance test passed, a project built, a Simulator launched, an application ran on a device.

Lived Evidence records what use revealed: the central gesture felt natural or forced, the intended routine did not survive contact, one screen contained the actual product, another person understood the application differently, or the Builder discovered a sharper need.

A compiler cannot prove usefulness. A feeling cannot prove signature validity. A passing test cannot prove demand. A truthful protocol never allows one kind of evidence to impersonate the other.

4.3 Rest

A Shot may rest. Rest is not failure, deletion, or death. It is the absence of a current intention to continue. A resting Shot remains whole and available.

TOHSENO must not optimize for the number of applications generated. It should optimize for honest creative loops completed. Without Contact, abundance becomes debris. With Contact, abundance becomes practice.

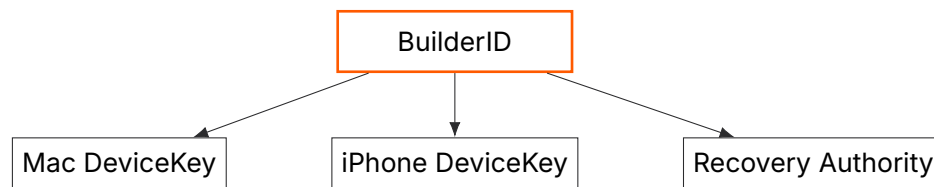
5 The BuilderID

5.1 A stable identity whose keys may change

A **BuilderID** is the stable protocol identity that controls one or more Shots.

It is not a username, an Apple ID, a seed phrase, a Mac, an iPhone, one permanent private key, or an account issued by a TOHSENO client. It is a public identifier for an authorization policy.

A useful mental model is a house whose address remains fixed while its keys and locks may change.



devices may be added or revoked;
the BuilderID remains the same

The Genesis implementation is expected to realize the BuilderID as a programmable smart account. Its address remains stable while authorized P-256 device public keys can be added, limited, or revoked. The abstract protocol does not depend on one account implementation; the public deployment identifies the exact verifier it uses.

5.2 The user's authority is not issued by the doorway

A compatible client may help a person establish a BuilderID. The client does not grant the identity and cannot revoke it by closing an account.

On first use, a device may generate a local signing key and calculate the BuilderID that will recognize it. Nothing needs to leave the machine until the Builder chooses a public action. A recovery mechanism may later restore authority or authorize a new device.

A 12-word mnemonic may be offered as one recovery method. It is not the BuilderID itself, and the protocol does not require a mnemonic for every Shot. Importing the same master secret into every generated application would create unacceptable risk.

The person controls the identity.
Devices carry replaceable authority.
Clients are replaceable interfaces.

5.3 Device authorization

Each physical device generates its own **DeviceKey**. A currently authorized device may authorize another device through a signed ceremony.

A possible pairing flow is:

1. A Mac creates a P-256 key and displays its public key in a short-lived QR challenge.
2. An authorized iPhone scans the challenge.
3. The person reviews the requested permissions and swipes to approve.
4. The iPhone signs an **AUTHORIZE_DEVICE** action.
5. Any relayer may carry the signed action to the BuilderID.
6. The BuilderID verifies the signature and adds the Mac key.

The iPhone never sends its private key to the Mac. The swipe is not a decorative gesture. It is the human authorization of a precise cryptographic action.

Apple produces the signature. The BuilderID establishes authority. TOHSENO defines what the action means. The public verifier enforces its consequence.

6 Installation Identity and Consentful Continuity

Builder identity and app-user identity must remain separate.

The BuilderID answers: *who controls this Shot?*

An **InstallationKey** answers: *which local installation is acting inside this application?*

Every TOHSENO application should be born with the same default identity law:

No username. No password. No mandatory TOHSENO account.
The installation creates an app-specific local key.

The InstallationKey is unique to that application installation. It is not a universal public identifier reused across every app. A person using Anky and a person using another TOHSENO app must not be silently correlatable merely because both apps share the same Fascia.

6.1 Continuity is a scoped proof, not a shared secret

When a person wants two applications to recognize a relationship, one installation may sign a narrow statement for the other.

For example:

I authorize this other application to verify that this installation completed today's writing ritual. This proof reveals no writing and expires after one use.

The receiving application verifies the proof. It does not receive a global account, the source application's database, the BuilderID, the person's other applications, or a reusable master secret.

This is **consentful continuity**: cooperation between personal applications without turning every application into a client of one central identity provider.

6.2 Passkeys are convenience, not membership

Passkeys may provide excellent continuity across Apple surfaces for a particular website or application. They may help with optional login, browser sessions, device pairing, or recovery.

They must not be the sole root of TOHSENO identity. A passkey is associated with a relying party. Making it mandatory would quietly privilege a domain, app bundle, provider, or Apple synchronization path.

A person who does not use iCloud must still be able to create locally, authorize devices, recover authority through another supported method, and continue a Shot through an independent client.

6.3 The possible innovation

TOHSENO does not invent P-256, deterministic derivation, mnemonic recovery, smart accounts, signatures, or content hashes.

Its proposed contribution is the composition of established cryptography into a default continuity layer for personal software:

- one durable BuilderID across replaceable clients and devices;
- one separate InstallationKey inside each app installation;
- no account screen for ordinary local use;
- no universal cross-app tracking identifier;
- explicit, narrow continuity proofs when cooperation is desired;
- an application lineage that survives the Factory that generated it.

The claim is not that the cryptography is new. The claim is that personal software can be born with ownership and consentful continuity as ordinary infrastructure rather than as a service added later.

7 The TOHSENO Apple Fascia

7.1 The known anatomy

Every compatible application shares a finite connective anatomy called the **TOHSENO Apple Fascia**.

The Fascia is not the visual design and not the purpose of the app. It is the shared structure that allows independent Factories, verifiers, nodes, and future intelligences to know what kind of organism they are looking at.

The Genesis Fascia defines these organs:

Application Shell	The Apple-native entry points, lifecycle, navigation, and platform boundaries.
Device Identity	The local installation key, signing interface, and consent rules.
Local Store	The local-first persistence interface and migration boundary.
Capability Blocks	Pinned, versioned infrastructure modules with declared interfaces.
Domain Module	The app-specific behavior through which the human intention becomes unique software.
Shot Provenance	The ShotID, lineage references, source commitments, and declared Factory information.
Distribution Configuration	The explicit state and requirements for local use, publication, and App Store distribution.

The domain may be radically different across applications. The connective structure remains legible.

7.2 Finite vocabulary, open expression

At any protocol expression, the Fascia vocabulary is finite, inspectable, and pinned. New capabilities may be introduced through a new versioned expression. Existing Shots continue to identify the exact Fascia under which they were created.

The first Fascia should be narrow. It should prefer Apple-native frameworks, local storage, explicit networking, and small capability surfaces over arbitrary infrastructure. Convention is not a failure of imagination. It is the common joint through which independent systems can cooperate.

Every app is sovereign, but no conforming app is unknowable.

7.3 The Markdown genome

A TOHSENO Factory may express each organ as a human-readable Markdown instruction paired with a machine-readable declaration.

A Genesis body may contain:

```
TOHSENO/  
  SHOT.md  
  FASCIA.md  
  IDENTITY.md  
  STORAGE.md  
  PRIVACY.md  
  DOMAIN.md  
  PROVENANCE.md  
  DISTRIBUTION.md  
  fascia.json  
  shot.json
```

The Markdown explains what the organ is and how an intelligence should grow or continue it. The machine declaration states the exact schema, version, interface, and commitment that a deterministic verifier must check.

The Markdown does not contain the organism. It contains instructions by which a compatible organism can be grown, inspected, and regenerated.

The app-specific generation material – a one-line intention, a master prompt, reference images, or other private context – may be called its **Genome**. The Genome is an implementation mechanism and remains private by default. The Fascia is the communal connective law.

8 The Canonical Objects

The TOHSENO system has five canonical objects. Their definitions must be understandable without ceremony.

The TOHSENO Protocol	Who owns a Shot, which signed actions count, how its history continues, and which facts may become public.
The TOHSENO Apple Fascia	The finite connective structure every compatible Apple application follows, regardless of the Apple surface on which its consequences are experienced.
The Protocol Schemas	The exact fields, allowed values, canonical byte encodings, and hashes that make TOHSENO objects unambiguous to machines.
The Deployed Registry Contract	The neutral public witness that verifies authorized actions and records the public state of Shots for one protocol deployment.
The Conformance Tests	The deterministic checks an app, Factory, node, or verifier must pass before it may truthfully claim compatibility.

No entry-point application is canonical. No model is canonical. No company database is canonical.

8.1 The smallest executable constitution

A white paper explains the law. A protocol requires the law to be mechanically reproducible.

For TOHSENO, the smallest executable constitution is the combination of:

- this constitutional paper;
- the schemas that define exact objects;
- the Fascia that defines known anatomy;
- the verifier that decides conformance;
- the registry that witnesses public authorization;
- the first Shot that proves the complete path exists.

The paper says *Builders own*. Signature verification makes that claim executable.

The paper says *any compatible Factory may continue a Shot*. Open schemas and conformance vectors make that claim executable.

The paper says *no doorway is official*. The absence of privileged contract callers makes that claim executable.

9 Signed Actions

9.1 A signature needs a shared meaning

Cryptography can prove that a key signed a digest. It cannot, by itself, say what the digest means.

TOHSENO defines a small vocabulary of typed actions. The Genesis vocabulary should remain minimal.

Identity actions:

- AUTHORIZE_DEVICE
- REVOKE_DEVICE
- CHANGE_RECOVERY

Shot actions:

- CREATE_SHOT
- EVOLVE_SHOT
- TRANSFER_SHOT
- PUBLISH_SHOT
- CLAIM_HANDLE
- ASSOCIATE_APPCOIN

Each action includes the identity or Shot it concerns, the exact payload commitment, replay protection, the relevant deployment, and an expiration where appropriate.

9.2 Four layers of meaning

1. **Cryptography** proves that a key signed a digest.
2. **The schema** proves which exact object produced that digest.
3. **The BuilderID** proves that the key had authority for the requested action.
4. **The registry** applies the public consequence defined by the protocol.

The specification defines the meaning.
 The schemas define the bytes.
 The BuilderID defines the authority.
 The contract enforces the public consequence.

9.3 The messenger is not the authority

Any party may carry a valid signed action to the public verifier:

- the Builder's client;
- Anky, Inc.'s relay;
- another Factory;
- a community node;
- a gas sponsor;
- the Builder directly.

The contract does not ask which doorway delivered the envelope. It asks whether the current controller authorized the action.

The messenger acquires no ownership by paying gas, relaying data, indexing a Shot, building source, or preserving public artifacts.

The contract does not care who delivered the envelope.
It verifies who signed the letter.

10 The Public Witness

10.1 The registry witnesses; it does not create

A Shot begins locally. Its private work, source, intention, and early Evolutions do not need a chain or network.

When the Builder chooses a public act, the registry may witness minimal facts:

- the ShotID;
- the current BuilderID controller;
- the current accepted public record commitment;
- the link to the previous public record;
- explicit public distribution state;
- explicit handle or appcoin associations;
- the event by which the state changed.

The registry does not need to store the source, private intention, app data, screenshots, model conversation, or full manifest. Those may remain local or live in separately chosen content-addressed storage.

A public record can attest that a Builder authorized a relation. It cannot make an incoherent app useful, prove that a person enjoyed it, or grant permission for a local Shot to exist.

10.2 No privileged registrar

A conforming registry must not require actions to arrive from an official TOHSENO server. It must not grant special rights to the deployer, Shot number one, Anky, Inc., the token issuer, or a preferred Factory.

The first deployer may remain visible in provenance. Historical authorship does not imply permanent authority.

The chain can remember the author without obeying the author.

10.3 The Genesis deployment

The first public registry is intended to be deployed on Robinhood Chain and to use its EVM environment and P-256 verification capability so Apple-native device signatures can be recognized without forcing an ordinary wallet experience.

This is a deployment choice, not the timeless definition of TOHSENO. The exact chain identifier, contract address, verification semantics, source hash, and deployment transaction belong in the Genesis Bundle's `DEPLOYMENT.json`.

The protocol name remains `tohseno`. Schemas, Fascia expressions, verifier releases, and registry deployments are versioned because exact machine expressions can contain defects or require explicit evolution. A later deployment cannot silently change what an earlier Shot record meant.

II Factories, Nodes, and Doorways

A conforming system distinguishes several roles.

Builder	Supplies intention, owns the app, authorizes external actions, encounters the result, and decides what happens next.
Factory	Turns intention into a conforming Apple application, invokes intelligence, protects rails, produces evidence, and leaves an independently operable repository.
Intelligence	Interprets meaning and performs open-ended implementation work. Powerful and replaceable.
Platform Toolchain	Swift, Xcode, Simulator, Apple signing, devices, and distribution machinery that establish Apple-specific facts.
Node	Participates in the network by indexing, mirroring, relaying, verifying, building, testing, routing, or sponsoring public work around Shots.
Registry	The neutral public witness for authorized Shot actions.
Reality	The context in which use confirms, resists, surprises, or redirects the intention.
Shot	The owned application lineage around which every other role is temporary.

II.1 Nodes index. Builders own.

A running TOHSENO instance may choose to participate as a node. It may announce which public artifacts it can serve, synchronize public headers, relay signed actions, receive feedback, verify conformance, or sponsor gas.

Node participation is additive. A Builder's local workshop must remain useful when every public node disappears.

Nodes may compete on service quality and policy. One may preserve source, another may build, another may route feedback, another may sponsor first publications. None becomes the owner by performing useful work.

II.2 Sponsorship and spam

Gas sponsorship can remove crypto friction without making the registry permissioned.

A node may pay for a valid action while applying its own sponsorship rules: quotas, conformance requirements, reputation, invitations, deposits, rate limits, or other evolving anti-spam methods. The node decides what it funds. The registry remains neutral about who is allowed to submit a correctly authorized action.

Private local creation remains free from registry spam because it does not require public registration.

11.3 One doorway may be best without becoming necessary

Anky, Inc. may build the most coherent first Factory, the easiest identity experience, the most useful default node, and the best managed services. People may prefer that doorway.

Preference is not permission.

The company competes on experience, execution, trust, and convenience. It does not win by making the protocol reject another conforming doorway.

12 Conformance

A protocol is not a brand. Compatibility must be decided by deterministic checks, not by a logo, company approval, model opinion, or marketing claim.

A conforming verifier should be capable of running locally and producing the same result across independent implementations.

12.1 Application conformance

A conforming application must, at minimum:

1. declare its ShotID and exact Fascia expression;
2. contain the required Fascia organs and interfaces;
3. match its declared source and manifest commitments;
4. create app-specific InstallationKeys rather than a universal tracking identifier;
5. remain useful locally without a mandatory TOHSENO account or server;
6. keep private data and raw intention local by default;
7. disclose every external dependency and irreversible action;
8. build and run through the standard declared Apple toolchain;
9. preserve the files required for an independent Factory to continue it.

12.2 Protocol record conformance

A conforming verifier must be able to check:

- canonical byte encoding;
- hashes and content commitments;
- signature validity;

- BuilderID authority;
- nonce and replay rules;
- previous-record lineage;
- allowed state transitions;
- declared schema and deployment identifiers.

12.3 Artificial intelligence cannot certify itself

Artificial intelligence may propose source, schemas, interfaces, tests, or Evolutions. It may not be the final judge of whether its own result conforms.

Artificial intelligence may propose.
 Deterministic machinery may prove.
 The Builder may authorize.
 Reality may answer.

A conformance tool should report passed, failed, unavailable, and unperformed checks separately. Unknown is not passed. Confidence is not evidence.

The command may be as simple as:

```
tohseno verify <shot>
```

The important property is not the command name. It is that another implementation can reproduce the judgment without asking an official service.

13 Local First and Privacy

13.1 The local artifact is real

A Shot begins in the Builder's local workshop. The repository, private Genome, app data, and unpublished Evolutions must not depend on a TOHSENO backend to remain coherent.

Local-first does not mean no network. A Shot may use a network because its purpose requires one. A Builder may choose cloud synchronization, hosting, collaboration, or managed publication. The constitutional rule is that no TOHSENO service becomes the hidden landlord holding the object together.

13.2 Different data belongs in different places

A conforming Apple application may use:

- local persistence for ordinary private app data;
- the Keychain and secure hardware for small credentials and device authority;
- optional private Apple synchronization when the Builder or user chooses it;
- the TOHSENO public network for deliberate public provenance;
- content-addressed storage for deliberately published artifacts.

No Apple ID is required at the protocol layer. Apple may require an Apple identity for platform signing, device installation, or App Store distribution. That is a separate relationship and must not become the definition of Builder ownership.

13.3 Private by default, public by declaration

The following remain private unless deliberately disclosed:

- raw intention and reference material;
- local app content;
- unpublished source;
- which apps a person uses;
- unapproved relationships between InstallationKeys;
- recovery material and private device keys.

Public provenance is a deliberate projection, not a dump of the Builder's thought process.

External and irreversible actions require specific, proximate, plain-language approval: spending, publishing, deploying, associating an appcoin, claiming a handle, transferring a Shot, submitting to the App Store, or disclosing user data.

TOHSENO tends ordinary creative friction toward zero. It does not tend responsibility toward zero.

14 Apple's Ground

TOHSENO is for the Apple ecosystem, on purpose.

The output is a native Apple application built with Apple's toolchain, not a provider-owned preview or a web surface disguised as the final object.

The Mac is the workshop.
Apple's other surfaces are where the work becomes life.

The iPhone may be the most intimate site of Contact for many applications. Other Shots may become true on iPad, Mac, Vision Pro, Watch, or future Apple surfaces. These are not separate protocols. They are surfaces on which one known application structure produces consequences.

The App Store is the summit of public Apple distribution. It is not the condition under which a Shot becomes real. A private application used by one person is still real software.

Apple's signing and distribution identity is not the BuilderID. A Builder may need an Apple ID or developer account to place a binary on a device or distribute publicly. That platform requirement cannot become protocol membership.

TOHSENO removes every landlord it can. It names the platform ground it deliberately keeps.

15 Handles and Economic Objects

15.1 A handle is an alias, not an identity

Human-readable handles are finite and useful. A public action may associate a handle such as **anky** with a ShotID.

The ShotID remains fundamental. A domain such as **tohseno.com/anky** is one gateway's rendering of the association. Another gateway may render the same Shot through another domain.

The domain does not grant identity to the Shot. The Shot gives the domain something to render.

15.2 An appcoin may be associated with a Shot

A Builder may sign an action associating an economic object with a Shot. The object may fund development, reward testing, coordinate participants, or express a community around the application.

The appcoin is not the Shot.

No token, asset, stake, payment, or financial relationship may become mandatory for ordinary local creation, ownership, use, Evolution, publication, or App Store distribution.

The economic layer may organize around the application.
The application must never be invented to justify the economic layer.

15.3 Shot number one

The first conforming TOHSENO application is intended to be registered as Shot number one.

Its public record may state:

- creator: JP Franetovic / Anky, Inc.;
- Factory: the first open-source TOHSENO implementation;
- handle: `tohseno`;
- associated appcoin: `$TOHSENO`;
- controller: the BuilderID designated by its creator;
- source and Genesis Bundle commitments.

These are provenance facts about one specific Shot. Being first confers no protocol privilege.

First in history. Equal under the protocol.

The token may point people toward the protocol. The protocol must not depend on the token.

16 Permanence and Versioning

16.1 The timeless name and the versioned expression

The root declaration may say:

`protocol: tohseno`

The exact machine expression must also identify its schema, Fascia, verifier, and public registry deployment.

This distinction preserves both the eternal center and technical honesty. The meaning of TOHSENO may remain stable while a schema, contract, or encoding receives an explicit successor. A later expression cannot silently rewrite an earlier Shot.

16.2 The Genesis Bundle

The first permanent release should contain, at minimum:

```
WHITEPAPER.pdf
WHITEPAPER.md
PROTOCOL.md
FASCIA/
SCHEMAS/
TEST_VECTORS/
CONFORMANCE.md
CONTRACTS/
DEPLOYMENT.json
SOURCE_COMMIT.txt
```

The bundle should be content-addressed and permanently published. The registry deployment should point to its exact commitment. A separate deployment record may contain final chain and contract facts if those cannot be known before the bundle is published.

The public point of entry may be one simple page linking the white paper, Genesis Bundle, registry, Shot number one, source repository, and conformance command.

No mythology is required. Only an inspectable chain of facts.

16.3 The paper should remain smaller than its consequences

This paper establishes what TOHSENO intends to preserve. It should not attempt to predict every node service, economic relationship, governance mechanism, application category, or future interface.

Future layers may preserve, transmit, verify, discover, build, test, fund, and coordinate around Shots. They must not redefine the Shot, capture its ownership, or make one doorway mandatory.

17 Constitutional Invariants

The irreducible commitments of TOHSENO are:

1. One coherent intention may become one independently operable Apple application.
2. A Shot is one application with a permanent identity, a known anatomy, and a signed history controlled by its Builder.
3. The Shot is the application-level object, not the prompt, model, Factory, build, account, handle, token, registry, or distribution channel around it.
4. A Shot begins locally. Public registration witnesses a deliberate public act; it does not grant permission for the Shot to exist.
5. A BuilderID is a stable public controller whose authorized device keys may change while the identity remains the same.
6. Builder identity, device authority, app Installation identity, and recovery authority are separate roles.
7. Every generated application creates an app-specific local InstallationKey. No universal identifier or mandatory TOHSENO account is required.
8. Cross-app continuity is explicit, scoped, and consented to. Apps may not silently correlate a person merely because they share the Fascia.
9. Every compatible Apple application follows the finite, inspectable, versioned TOHSENO Apple Fascia.
10. Artificial intelligence may express the application without bounds inside the Fascia. Deterministic machinery verifies the invariants.
11. The specification defines the meaning of signed actions. Schemas define their bytes. BuilderIDs define authority. Contracts enforce public consequences.
12. Anyone may relay a valid signed action. Relaying, sponsoring, indexing, building, or mirroring a Shot does not confer ownership.
13. No privileged registrar, official client, founder key, company server, token, or node may be required for ordinary creation, ownership, use, or continuation.
14. The repository is ejectable from birth. A Builder can build, inspect, modify, and continue the application without TOHSENO permission.
15. Raw intention, local app data, recovery material, and unapproved identity relationships remain private by default.
16. Apple signing and distribution identities are separate from TOHSENO protocol identity. Apple ID is not protocol membership.
17. The Mac is the workshop. Apple surfaces are where the consequences are lived. The App Store is a distribution summit, not the definition of reality.
18. A handle or appcoin may be explicitly associated with a Shot. Neither becomes the Shot or a requirement for

its life.

19. The first TOHSENO application may be Shot number one and the first Anky, Inc. doorway may be the most widely used. Neither receives special protocol rights.
20. Nodes index. Builders own.

18 Conclusion

The scarce thing in software is no longer the ability to produce code. It is the ability to carry intention across intelligence, machines, devices, companies, and time without losing ownership or meaning.

TOHSENO begins with one human intention and gives it an Apple-native body. The application is born inside a known Fascia. Its ShotID preserves continuity. Its BuilderID preserves authority across replaceable device keys. Its InstallationKey permits local use without an account. Explicit continuity proofs allow applications to cooperate without silently collapsing a person into one universal identity. Deterministic verification keeps the promises exact while artificial intelligence expands what may be expressed.

The first Factory can disappear. The Shot remains.

The first model can be replaced. The Shot remains.

A device can be lost and revoked. The BuilderID remains.

A node can stop serving. The Builder remains the owner.

A token can rise or fall. The application remains real.

Anky, Inc. can build the first doorway. The protocol refuses to make that doorway necessary.

A Shot is an Apple app with a permanent identity,
a known anatomy, and a history controlled by its Builder.

TOHSENO is the open protocol through which compatible intelligences and machines may create, understand, verify, and continue that life.

Intelligence imagines without bounds; machines keep the promises exact.

Nodes index. Builders own.

Give every idea a shot. Let reality continue it. Take another one.

Glossary

Apple Fascia

The finite connective structure every compatible TOHSENO Apple application follows.

Builder

The human or controlling party that supplies intention, owns the application, authorizes external actions, encounters its consequences, and decides what happens next.

BuilderID

A stable public identity for an authorization policy controlling Shots; its authorized device keys may change without changing the identity.

Conformance

Deterministic agreement with the declared schemas, Fascia, signatures, lineage, interfaces, and independence requirements of a TOHSENO expression.

Consentful Continuity

A scoped, explicit, signed relationship through which one app installation proves a particular fact or authority to another without revealing a universal identity or shared secret.

Contact

Human encounter with a functioning application in a context where its central behavior can become true or false.

DeviceKey

A replaceable signing key generated and held by one physical device and authorized to perform specified actions for a BuilderID.

Evolution

A meaningful continuation of the same Shot, authorized by its controller and linked to its previous accepted record.

Factory

A conforming system that turns intention into an independently operable Apple application, invokes intelligence, protects protocol rails, and produces verifiable evidence.

Genome

The private or app-specific instruction material from which a Factory grows one expression of an application. The Genome is not the communal Fascia.

InstallationKey

An app-specific local key representing one installation of one application. It is separate from the BuilderID and from other applications' installation identities.

Node

A participant that may index, mirror, relay, verify, build, test, route, or sponsor public activity around Shots without becoming their owner.

Public Witness

The deployed registry that verifies authorized public actions and records minimal public Shot state.

Recovery Authority

A supported method for restoring or changing control of a BuilderID without treating one lost device key as the permanent identity.

Shot

One Apple application with a permanent identity, a known anatomy, and a signed history controlled by its Builder.

Shot Genesis

The first accepted record of a Shot.

ShotID

The stable opaque identifier that remains constant across Evolutions of the same Shot.

TOHSENO

The open protocol for creating and evolving owned Apple applications through shared structure, portable identity, signed lineage, local-first independence, and deterministic conformance.

END OF GENESIS PAPER

Give every idea a shot. Let reality continue it. Take another one.